

Reviewer Pack — Minimal Number of Galois Automorphisms and Resolution Proof ...

Entry aacef77026ad · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 06:02:29 UTC

Minimal Number of Galois Automorphisms and Resolution Proof Width Inequality

Entry ID: aacef77026ad **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 06:02:29 UTC

1. Conjecture

Field A (mathematical branch): Galois Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given CNF φ with n variables, the minimal number of automorphisms in its associated Galois group $G(\varphi)$ is upper-bounded by a function $g(n)$, such that the resolution proof width $w(\varphi)$ of φ satisfies $w(\varphi) \leq g(n)^2$.

Rationale (proposer's reasoning):

Galois theory provides a way to study symmetry and structure in polynomials, which might be reflected in the complexity of resolution proofs. A larger number of automorphisms suggests a more complex symmetry, potentially leading to a higher proof width.

Taxonomy category: Algebraic Structures in Complexity Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 309dcd0d9a71bdd3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated CNFs φ with n variables, the minimal number of automorphisms in $G(\varphi)$ is $\leq g(n)$ and the resolution proof width $w(\varphi)$ is $\leq g(n)^2$ for at least 80% of the seeds and with an aggregate mean metric ≤ 3 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Minimal number of automorphisms AND Galois theory AND resolution proof complexity - Galois group automorphism count INCLUSIVE resolution proof width - Resolution proof width inequality related to Galois theory automorphisms

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10 * n): # Generate 10 clauses per variable on average
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def galois_group(cnf):
        # Simplified version for demonstration; actual implementation needed
        return len(cnf) # Placeholder

    def resolution_width(cnf):
        # Simplified version for demonstration; actual implementation needed
        return len(cnf) ** 2 # Placeholder

    n = random.randint(5, 40)
    cnf = generate_cnf(n)

    num_automorphisms = galois_group(cnf)
    proof_width = resolution_width(cnf)

    return {
        "metric_name": "resolution_proof_width",
        "metric_value": proof_width,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": proof_width <= num_automorphisms ** 2,
```

```

        "counterexample": "" if conjecture_holds else f"CNF with {n} variables and width {proof_width}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in enumerate(results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b096287d.py", line 56, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b096287d.py", line 48, in run_trial
    "counterexample": "" if conjecture_holds else f"CNF with {n} variables and width {proof_width}"
                          ~~~~~^
NameError: name 'conjecture_holds' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-examine the test code for errors and ensure that it can run to completion without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14080
2	preregistration	ollama_remote	glm4:latest	0	0	10362
3	novelty	ollama_remote	glm4:latest	0	0	9997
4	novelty	ollama_remote	glm4:latest	0	0	12500
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13918
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8125
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6645
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6727
9	judge	ollama_remote	glm4:latest	0	0	14672

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97027 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/aacef77026ad.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aacef77026ad.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aacef77026ad.tar.gz` (if generated)